AIOps Module 1: Experiment Management & Reproducibility

Khaja Raisser (DA24B015) | Partner: Mudda Manikanta Pruthvi Raj
Indian Institute of Technology Madras | August 2026

1. Technical Debt Diagnosis

(a) Entanglement (CACE): Modifying the delivery-time feature's rounding logic unexpectedly degraded the unrelated favorite-restaurants feature. In ML systems, the learned weight matrix couples all input features, so changing one feature's preprocessing shifts the model's behavior on unrelated outputs.

(b) Undeclared Consumers: The marketing team was silently reading the model's raw output table without the ML team's knowledge, creating an invisible downstream dependency. Any schema or distribution change by the ML team would break the marketing pipeline without warning.

(c) Configuration and Glue-Code Debt (Pipeline Jungle): The training pipeline is a chain of 14 undocumented shell scripts with no orchestration tool. No parameterization, no reproducibility, and tribal knowledge is required to operate it.

Mitigation for (c): Replace the shell scripts with an **MLflow Project**. An `MLproject` file defines named entry points with declared parameters and types, paired with a `python_env.yaml` for pinned dependencies. The pipeline becomes executable via `mlflow run . -P lr=0.01`, and each invocation auto-creates a tracked MLflow run, centralizing all parameters and metrics.

2. MLflow Experiment Comparison

Six experiments trained a two-layer MLP (sklearn `MLPClassifier`) on MNIST, varying learning rate and hidden layer architecture. All runs used `batch_size=200`, `seed=42`, and 20 epochs.

Run	Learning Rate	Hidden Layers	Batch Size	Accuracy
1	0.001	(256, 128)	200	0.9751
2	0.001	(128, 64)	200	0.9725
3	0.01	(128, 64)	200	0.9694
4	0.0001	(128, 64)	200	0.9627
5	0.01	(256, 128)	64	0.9608
6	0.001	(64, 32)	200	0.9603

Best run: `lr=0.001` with hidden layers `(256, 128)`, achieving 97.51% accuracy. The larger network capacity combined with a moderate learning rate produced the best generalization.

Overfitting evidence: Training loss decreased steadily from 0.39 to 0.01 across 20 epochs, while validation accuracy plateaued around epoch 5 at approximately 96.6% and improved only marginally thereafter. This widening gap is a characteristic indicator of overfitting.

Dominant hyperparameter: Learning rate had the larger effect. Holding architecture constant at (128, 64), varying the learning rate caused up to a 1% accuracy swing. Architecture changes showed a comparable range but with a smoother gradient across configurations.

3. DVC Data Versioning and Rollback

DVC was initialized to track a CSV file of image filenames. Version 1 contained 1800 rows; after appending entries from a supplementary label set, version 2 contained 2801 rows. Both versions were pushed to an SSH-based DVC remote. Rollback was verified by running `git checkout v1` followed by `dvc checkout`, restoring the file to exactly 1800 rows (`wc -l`: 1801 including the header). Returning to `master` and executing `dvc checkout` restored version 2 (`wc -l`: 2802). This confirms that DVC's content-addressed storage correctly restores data files when the `.dvc` pointer is reverted.

4. End-to-End Reproducibility

Partner A: Trained an MLP on the DVC-tracked MNIST dataset. Logged all parameters, the final accuracy metric (0.9782), and a `git_commit` tag in MLflow. The training code and `.dvc` pointer were committed together in a single Git commit. The model was registered as `my-classifier` in the MLflow Model Registry and transitioned to *Staging*.

Partner B: Cloned the repository, created the environment from `environment.yml`, pulled the dataset via DVC over SSH, and re-ran the training script. Achieved accuracy of 0.9784 (difference of 0.0002, within the 0.005 tolerance). The minor discrepancy is attributable to floating-point arithmetic differences across hardware. A reproducibility note confirming the match was logged in MLflow.

The full protocol (Git + DVC + MLflow + pinned environment) successfully enabled independent reproduction.